ESE5023 Assignment 02

姓名：吴昱萱　学号：12632745

我参考了 pandas 文档中关于 groupby().transform() 和月份处理的说明，以及 Matplotlib 文档中关于 errorbar() 的说明。第 4 题我采用了本人课题组的一部分地下水数据。选取数据也同步上传进仓库。

Q1

1.1 Total deaths per country, top 20

按 Country 分组，对 Deaths 求和，最后删除缺失结果、按降序排序输出前 20 个.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

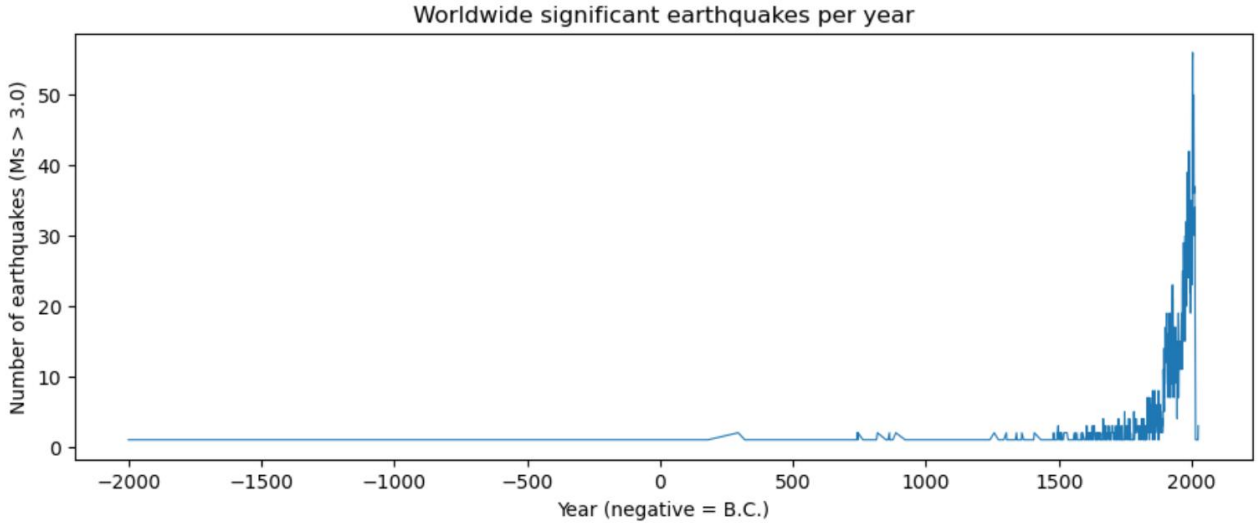
# Q1
# Change to the name of the TSV file you downloaded from NOAA
EQ_FILE = r"C:\Users\Lenovo\Downloads\earthquakes-2026-10-14_20-30-46_+0800.tsv"

Sig_Eqs = pd.read_csv(EQ_FILE, sep="\t")
Sig_Eqs.columns = [c.strip() for c in Sig_Eqs.columns]
print(Sig_Eqs.columns.tolist())
```

```
['Search Parameters', 'Id', 'Year', 'Mo', 'Dy', 'Hr', 'Mn', 'Sec', 'Tsu', 'Vol', 'Country', 'Area', 'Region', 'Location Name', 'Latitude', 'Longitude',
'Focal Depth (km)', 'Mag', 'Mw', 'Ms', 'Mb', 'Ml', 'Mfa', 'Unk', 'MMI Int', 'Deaths', 'Death Description', 'Missing', 'Missing Description', 'Injurie
s', 'Injuries Description', 'Damage ($Mil)', 'Damage Description', 'Houses Destroyed', 'Houses Destroyed Description', 'Houses Damaged', 'Houses Damage
d Description', 'Total Deaths', 'Total Death Description', 'Total Missing', 'Total Missing Description', 'Total Injuries', 'Total Injuries Descriptio
n', 'Total Damage ($Mil)', 'Total Damage Description', 'Total Houses Destroyed', 'Total Houses Destroyed Description', 'Total Houses Damaged', 'Total H
ouses Damaged Description']
```

1.2 Yearly number of earthquakes with Ms > 3.0

先用 Ms > 3.0 筛选事件，再按 Year 分组并用 size() 计算每年的记录条数。负年份表示公元前。折线图以 Year 为横轴、记录数为纵轴。对 1900 年以来的年记录数进行线性回归，斜率为每年增加 0.169 次，$p = 3.54 \times 10^{-10}$。



1500 年以前，记录到的重大地震数量接近于零，此后逐渐增加，并在约 1900 年后急剧上升，于 2000 至 2010 年间达到峰值。这种明显的增长主要源于探测和记录能力的提升，而非地震本身变多。末期的下降可能反映了近期事件编录的不完整。

1.3 CountEq_LargestEq

CountEq_LargestEq(country) 先取指定国家的记录，len(sub) 给出该国地震条数；随后删去 Ms 缺失的行，用 idxmax() 找到 Ms 最大的一行，再从该行取 Year、Mo、Dy 和 Location Name。函数返回地震次数、日期和地点。

```
[6]: #1.3 CountEq_LargestEq
def CountEq_LargestEq(country, df=Sig_Eqs, mag_col=MAG):
    """Return (number of earthquakes, date of largest quake, location of largest quake)."""
    sub = df[df["Country"] == country.upper()]
    n = len(sub)
    sub_m = sub.dropna(subset=[mag_col])
    if sub_m.empty:
        return n, None, None
    row = sub_m.loc[sub_m[mag_col].idxmax()]

    def g(c):
        v = row.get(c, np.nan)
        return int(v) if pd.notna(v) else None

    date = f"{g('Year')}-{g('Mo')}-{g('Dy')}"
    return n, date, row.get("Location Name")

rows = []
for c in Sig_Eqs["Country"].dropna().unique():
    n, d, loc = CountEq_LargestEq(c)
    rows.append((c, n, d, loc))

result = (pd.DataFrame(rows, columns=["Country", "N_earthquakes", "Largest_date", "Largest_location"])
          .sort_values("N_earthquakes", ascending=False)
          .reset_index(drop=True))
print(result.to_string())
```

```
98                                VIETNAM             6       1983-6-24                                VIETNAM
99                                  GHANA             6       1939-6-22                          GHANA:  ACCRA
100                         NORTH KOREA               6       1668-7-31                 NORTH KOREA:  YELLOW SEA
```

最后设计循环，代码遍历 Country 的唯一值，把每国的函数结果放入 rows，创建结果表后按 N_earthquakes 降序输出。若一国所有 Ms 都缺失，函数返回次数和两个 None。

Q2

数据处理与筛选：

TMP 形如 “+0250,1”。代码按逗号拆成数值 T 与质量码 Q，用 pd.to_numeric 将 T 转成数字；无法转换的文本变为缺失值。

筛选与单位换算：代码剔除 +9999 缺失温度，只保留质量码 0、1、4、5，再将 T 除以 10，得到摄氏度。该步骤保留 234,864 条、剔除 810 条记录。

```
# Q2
# Give the full path
SZ_FILE = r"E:\Baoan_Weather_1998_2022\Baoan_Weather_1998_2022.csv"
sz = pd.read_csv(SZ_FILE, low_memory=False)
print(sz.columns.tolist())
print(sz[["TMP"]].head())
```

```
['STATION', 'DATE', 'SOURCE', 'REPORT_TYPE', 'CALL_SIGN', 'QUALITY_CONTROL', 'AA1', 'AA2', 'AA3', 'AG1', 'AJ1', 'AW1', 'AY1', 'AY2', 'AZ1', 'CALL_SIGN.
1', 'CIG', 'DEW', 'ED1', 'EQD', 'GA1', 'GA2', 'GA3', 'GA4', 'GA5', 'GE1', 'GF1', 'HL1', 'IA1', 'IA2', 'KA1', 'KA2', 'MA1', 'MD1', 'ME1', 'MW1', 'MW2',
'MW3', 'OA1', 'OC1', 'OD1', 'OD2', 'QUALITY_CONTROL.1', 'REM', 'REPORT_TYPE.1', 'SA1', 'SLP', 'SOURCE.1', 'TMP', 'UA1', 'UG1', 'VIS', 'WG1', 'WND']
       TMP
0  +0186,1
1  +0220,1
2  +0240,1
3  +0221,1
4  +0240,1
```

```
# Filter TMP (ISD format, POS 88-93)
# TMP looks like "+0250,1": air temperature in tenths of °C, then a quality code.
#   +9999 = missing
#   quality codes 0, 1, 4, 5 = passed checks (kept)
#   quality codes 2, 3, 6, 7, 9 = suspect / erroneous / missing (dropped)
tcol = sz["TMP"].astype(str).str.split(",", expand=True)
sz["T"] = pd.to_numeric(tcol[0], errors="coerce")
sz["Q"] = tcol[1]

n_before = len(sz)
good = (sz["T"] != 9999) & (sz["Q"].isin(["0", "1", "4", "5"]))
sz = sz[good].copy()
sz["T"] = sz["T"] / 10.0                    # scale factor 10 -> °C
print(f"Kept {len(sz)} of {n_before} records ({len(sz)/n_before:.1%})")
```

```
Kept 234864 of 235674 records (99.7%)
```

对月平均气温进行回归，得到约 +0.387 °C/十年，p = 0.339，表明该站点记录的数据整体升温，趋势较小。

```
# Time index
time_col = "DATE" if "DATE" in sz.columns else sz.columns[0]
sz["time"] = pd.to_datetime(sz[time_col])
sz = sz.set_index("time").sort_index()

# Monthly mean and plot
monthly = sz["T"].resample("MS").mean()

m = monthly.dropna()
x = (m.index - m.index[0]).days / 365.25      # time in years
res = stats.linregress(x, m.values)

fig, ax = plt.subplots(figsize=(11, 4))
ax.plot(monthly.index, monthly.values, lw=1, label="Monthly mean")
ax.plot(m.index, res.intercept + res.slope * x, "r--",
        label=f"Linear trend: {res.slope*10:.2f} °C/decade")
ax.set_xlabel("Time")
ax.set_ylabel("Monthly mean air temperature (°C)")
ax.set_title("Baoan International Airport, Shenzhen (1998-2022)")
ax.legend()
plt.show()

print(f"Trend = {res.slope:.4f} °C/yr, r = {res.rvalue:.3f}, p = {res.pvalue:.3g}")
```

Q3

parse_dates 将 ISO_TIME 解析为时间；na_values 将未命名风暴等占位符转为缺失。随后再次把 WMO_WIND、LAT、LON 转为数值。

```
# Q3
IB_FILE = r"E:\ibtracs.ALL.list.v04r00\ibtracs.ALL.list.v04r00.csv"
# skiprows=[1, 2] skips the units row(s) under the header, so numeric columns parse as numbers.
# na_values turns placeholder names into NaN.
df = pd.read_csv(IB_FILE,
                 usecols=range(17),
                 skiprows=[1, 2],
                 parse_dates=["ISO_TIME"],
                 na_values=["NOT_NAMED", "NAME", " "])

# Make sure the numeric columns are numeric
for c in ["WMO_WIND", "LAT", "LON"]:
    df[c] = pd.to_numeric(df[c], errors="coerce")

print(df.shape)
print(df.head())
```

```
C:\Users\Lenovo\AppData\Local\Temp\ipykernel_40592\3444993216.py:4: DtypeWarning: Columns (5,12) have m
t low_memory=False.
  df = pd.read_csv(IB_FILE,
(707176, 17)
              SID  SEASON  NUMBER BASIN SUBBASIN NAME            ISO_TIME  \
0  1842298N11080    1842       1    NI       BB  NaN 1842-10-25 06:00:00
1  1842298N11080    1842       1    NI       BB  NaN 1842-10-25 09:00:00
2  1842298N11080    1842       1    NI       BB  NaN 1842-10-25 12:00:00
3  1842298N11080    1842       1    NI       BB  NaN 1842-10-25 15:00:00
4  1842298N11080    1842       1    NI       AS  NaN 1842-10-25 18:00:00

  NATURE      LAT      LON  WMO_WIND  WMO_PRES WMO_AGENCY TRACK_TYPE  \
0     NR  10.8709  79.8265       NaN       NaN        NaN       main
1     NR  10.8431  79.3524       NaN       NaN        NaN       main
2     NR  10.8188  78.8772       NaN       NaN        NaN       main
3     NR  10.8000  78.4000       NaN       NaN        NaN       main
4     NR  10.7884  77.9194       NaN       NaN        NaN       main
```

3.1 与 3.2

按 SID 分组，对 NAME 和 SEASON 取首个值，对 WMO_WIND 取最大值；按最大风速排序，前 10 条用于名称清单，前 20 条用于风速清单。这样每场风暴只出现一次。Patricia 的最大 WMO 风速为 185 节。

```
# 3.1 Names of the 10 largest hurricanes by wind speed
storm = df.groupby("SID").agg(NAME=("NAME", "first"),
                              SEASON=("SEASON", "first"),
                              MAX_WIND=("WMO_WIND", "max"))
top10 = storm.sort_values("MAX_WIND", ascending=False).head(10)
print(top10)
print("\nNames:", top10["NAME"].tolist())
```

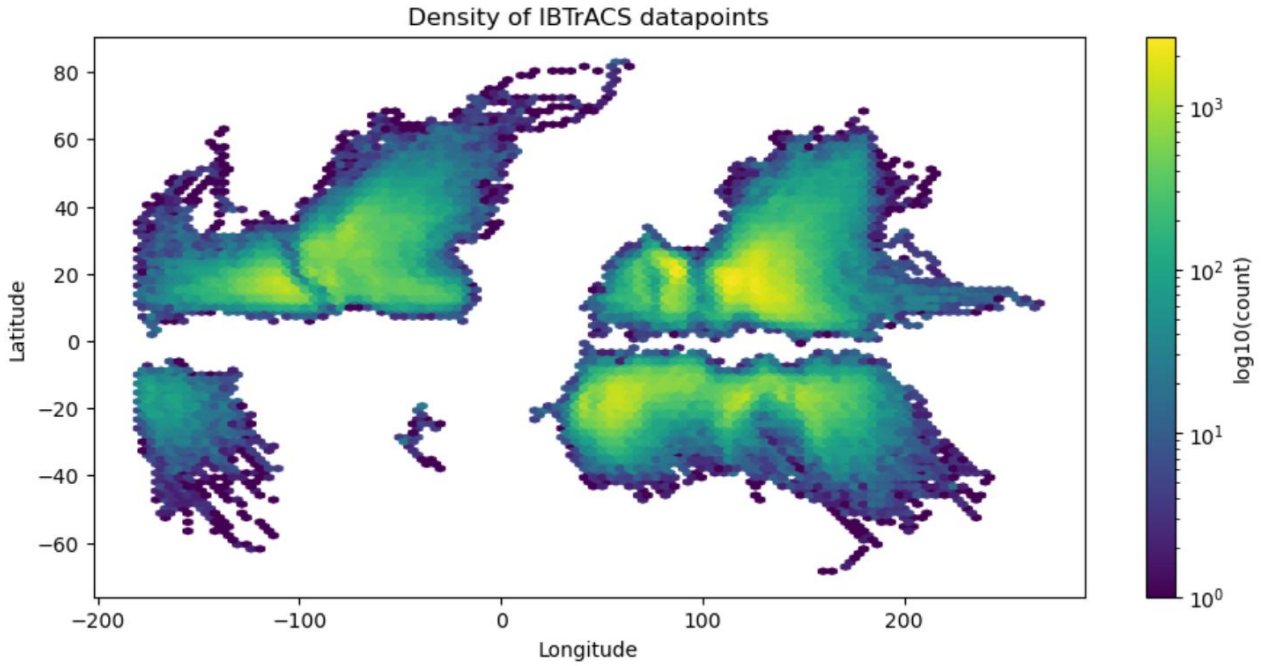
3.3

代码按 BASIN 分组计数，但 pandas 默认丢弃 BASIN 缺失的行。

3.4

hexbin 以 LON 和 LAT 为坐标，将相近位置的点放进六边形网格；bins="log" 令颜色使用对数尺度，使高密度和低密度海域都可辨认。

```
# 3.4 Hexbin plot of datapoint locations
fig, ax = plt.subplots(figsize=(11, 5))
hb = ax.hexbin(df["LON"], df["LAT"], gridsize=100, bins="log", cmap="viridis", mincnt=1)
fig.colorbar(hb, label="log10(count)")
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.set_title("Density of IBTrACS datapoints")
plt.show()
```

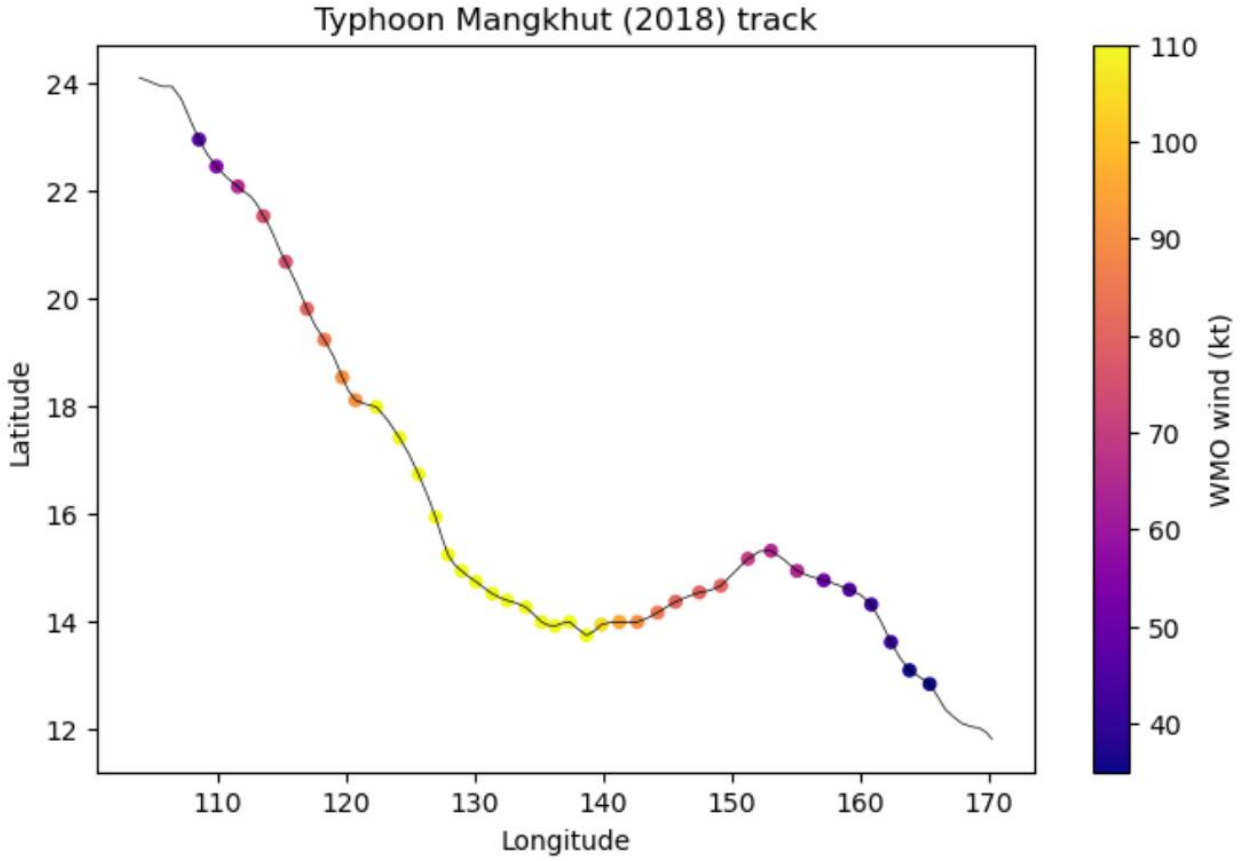


3.5

同时筛选 NAME 为 MANGKHUT、观测年份为 2018 的记录；折线连接经纬度点，散点颜色表示该时刻的 WMO_WIND。图中共有 99 个点。

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(mk["LON"], mk["LAT"], "k-", lw=0.5)
sc = ax.scatter(mk["LON"], mk["LAT"], c=mk["WMO_WIND"], cmap="plasma", s=20)
fig.colorbar(sc, label="WMO wind (kt)")
ax.set_xlabel("Longitude")
ax.set_ylabel("Latitude")
ax.set_title("Typhoon Mangkhut (2018) track")
plt.show()
```

```
Mangkhut datapoints: 99
```



3.6

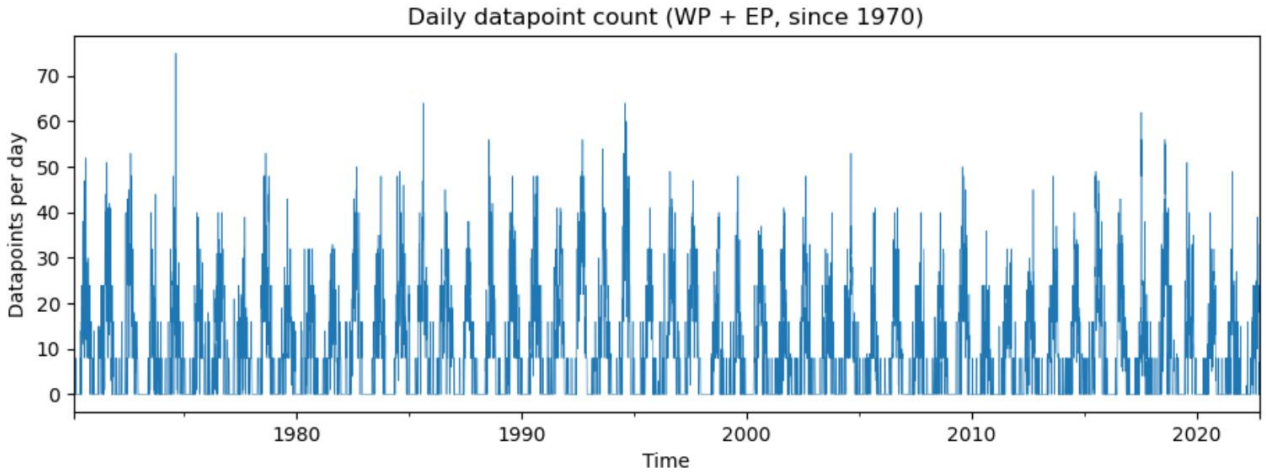
筛选 1970 年及之后、海盆为 WP 或 EP 的记录后，共保留 176,352 个观测点。图中的每日数值是观测记录条数，不是当天活动风暴的场数。

3.7

把 ISO_TIME 设为索引，通过 resample("D").size() 统计每天的观测记录数，未有记录的日期计为零。这里统计的是观测点，不是独立风暴数。

```
# 3.7 Number of datapoints per day
daily = dfw.set_index("ISO_TIME").resample("D").size()

fig, ax = plt.subplots(figsize=(11, 3.5))
daily.plot(ax=ax, lw=0.4)
ax.set_xlabel("Time")
ax.set_ylabel("Datapoints per day")
ax.set_title("Daily datapoint count (WP + EP, since 1970)")
plt.show()
```

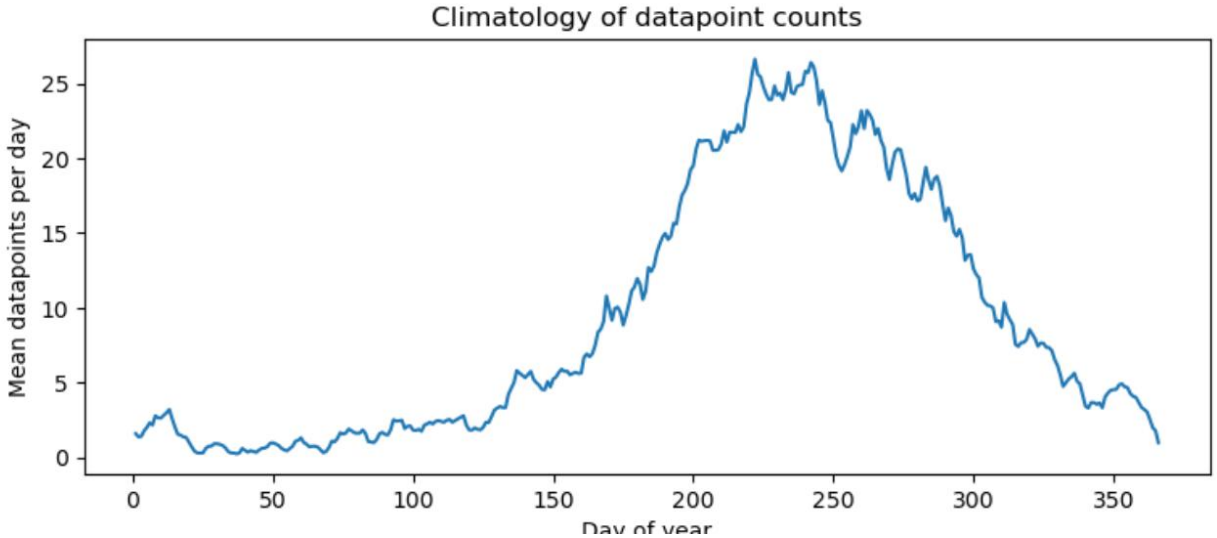


3.8

对 daily 按 dayofyear 分组并取多年平均，得到每个年内日序的“气候态”。

```
# 3.8 Climatology as a function of day of year
clim = daily.groupby(daily.index.dayofyear).mean()

fig, ax = plt.subplots(figsize=(9, 3.5))
clim.plot(ax=ax)
ax.set_xlabel("Day of year")
ax.set_ylabel("Mean datapoints per day")
ax.set_title("Climatology of datapoint counts")
plt.show()
```



3.9

把每天的计数减去相应日序的多年平均值，得到每日距平。正值表示当天观测点数高于该日序的平均水平。

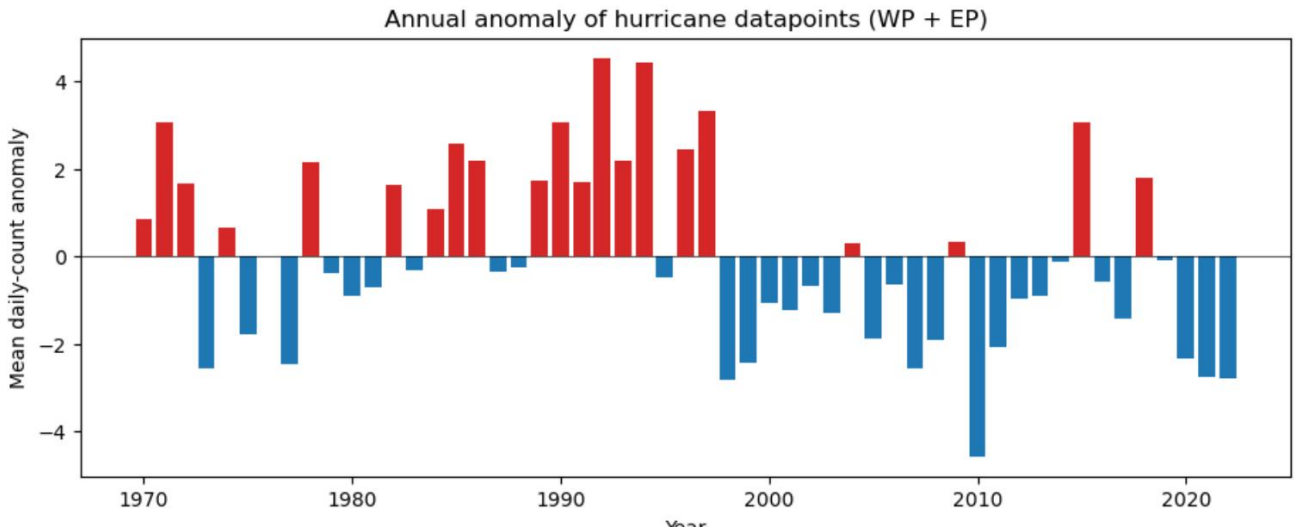
3.10

通过 resample("YS").mean() 求每年平均每日距平，并以距平绝对值大于该年序列一个标准差列出突出年份。1992 年和 1994 年偏高，2010 年偏低。

```python
# 3.10 Resample anomaly to annual resolution
anom_y = anom.resample("YS").mean()

fig, ax = plt.subplots(figsize=(11, 4))
ax.bar(anom_y.index.year, anom_y.values,
       color=np.where(anom_y.values > 0, "tab:red", "tab:blue"))
ax.axhline(0, color="k", lw=0.5)
ax.set_xlabel("Year")
ax.set_ylabel("Mean daily-count anomaly")
ax.set_title("Annual anomaly of hurricane datapoints (WP + EP)")
plt.show()

thr = anom_y.std()
print("Years with |anomaly| > 1 std:")
print(anom_y[anom_y.abs() > thr].round(2).to_string())
```



Q4

4.1 读取监测井文件，跳过前四行表头，向下填充井号，解析采样日期，并将测量列转成数值。随后删除日期或地下水位缺失的记录，保留 300—600 米范围内的水位。141 行原始记录中有 128 行通过清理；其中至少采样六次的 14 口井共有 110 次观测，用于时间序列和统计分析。

4.2 循环 14 口井，按日期绘制各井地下水位，比较同一口井随时间的变化。

```
FILE = r"C:\Users\Lenovo\Desktop\4.xlsx"

raw = pd.read_excel(FILE, header=None)     # the sheet has 4 header rows (names, sub-names, units, limits)
data = raw.iloc[4:].copy()                 # real data start at row 5

# pick columns by position and give them English names
cols = {0: "well", 1: "date", 10: "well_depth_m", 11: "depth_to_water_m", 12: "water_level_m",
        13: "water_temp_C", 16: "pH", 17: "ORP_mV", 18: "TDS_mgL", 20: "EC_uScm", 22: "DO_pct"}
df = data[list(cols)].rename(columns=cols)

# the well ID is only written on the first row of each well -> fill down
df["well"] = df["well"].ffill().astype(str)

# dates look like "2025.7.7"
df["date"] = pd.to_datetime(df["date"].astype(str).str.strip(), format="%Y.%m.%d", errors="coerce")

# text such as "14.57mS/cm" or "/" becomes NaN
for c in df.columns.drop(["well", "date"]):
    df[c] = pd.to_numeric(df[c], errors="coerce")

n0 = len(df)
df = df.dropna(subset=["date", "water_level_m"])                # missing values
df = df[df["water_level_m"].between(300, 600)]                  # physically implausible values
df["pH"] = df["pH"].where(df["pH"].between(0, 14))              # bad pH -> NaN
df = df.sort_values(["well", "date"]).reset_index(drop=True)
print(f"Kept {len(df)} of {n0} rows after cleaning")
print(df.isna().sum())

# wells with at least 6 visits are used for time series and statistics
counts = df.groupby("well").size()
main = counts[counts >= 6].index
sub = df[df["well"].isin(main)].copy()
print("Wells used:", list(main))
```

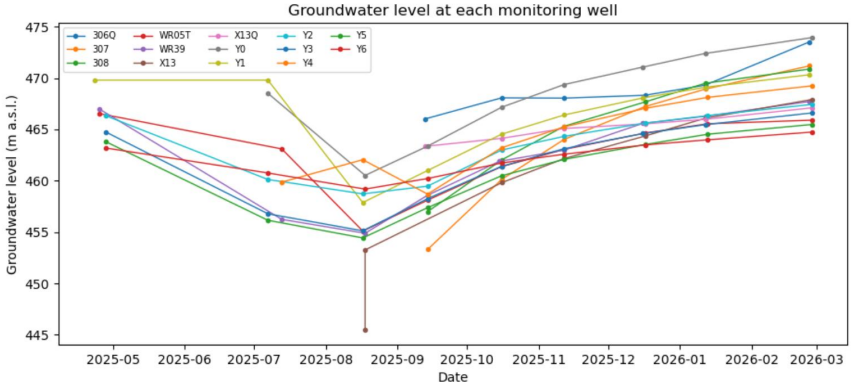
```
Kept 128 of 141 rows after cleaning
```

4.2 每次观测减去该井的平均水位，得到井内距平；再按月份汇总均值、标准差和样本数，并用误差棒显示月平均值及一个标准差。

```
# 4.2 Time series
fig, ax = plt.subplots(figsize=(11, 4.5))
for w in main:
    s = sub[sub["well"] == w]
    ax.plot(s["date"], s["water_level_m"], marker="o", ms=3, lw=1, label=w)
ax.set_xlabel("Date"); ax.set_ylabel("Groundwater level (m a.s.l.)")
ax.set_title("Groundwater level at each monitoring well")
ax.legend(ncol=5, fontsize=7)
plt.show()

# anomaly relative to each well's own mean, averaged by month
sub["anom"] = sub["water_level_m"] - sub.groupby("well")["water_level_m"].transform("mean")
sub["month"] = sub["date"].dt.to_period("M").dt.to_timestamp()
mm = sub.groupby("month")["anom"].agg(["mean", "std", "count"])

fig, ax = plt.subplots(figsize=(11, 4))
ax.errorbar(mm.index, mm["mean"], yerr=mm["std"], fmt="o-", capsize=3)
ax.axhline(0, color="k", lw=0.5)
ax.set_xlabel("Month"); ax.set_ylabel("Level anomaly (m)")
ax.set_title("Monthly mean groundwater-level anomaly (error bar = 1 std across wells)")
plt.show()
```



4.3

做了七项检查：先用 describe()概括 110 条记录的平均值、标准差和取值范围；再对每口井分别拟合水位随时间变化的直线，并检验 14 口井的平均变化斜率是否为零；接着比较 7–9 月与 12–2 月的水位偏差，计算水位与水温的 Spearman 相关系数，检验水位偏差是否符合正态分布，找出偏离平均值超过 3 个标准差的记录，最后用 Kruskal–Wallis 检验比较各井的水位分布。结果显示，平均水位为 463.79 m，各井的平均斜率约为 +1.18 m/30 天，夏季偏差低于冬季，水位与水温呈负相关，并找到 1 条可能的离群记录。解释检验的 p 值时要谨慎，因为同一口井的多次测量可能彼此相关，而部分检验把它们当作独立记录。

```
# 3) seasonal contrast: Jul-Sep (low) vs Dec-Feb (high), Mann-Whitney U test
low = sub[sub["date"].dt.month.isin([7, 8, 9])]["anom"]
high = sub[sub["date"].dt.month.isin([12, 1, 2])]["anom"]
u = stats.mannwhitneyu(low, high)
print(f"3) Mean anomaly Jul-Sep = {low.mean():.2f} m, Dec-Feb = {high.mean():.2f} m, p = {u.pvalue:.2e}\n")
```

```
3) Mean anomaly Jul-Sep = -4.76 m, Dec-Feb = 3.42 m, p = 1.13e-12
```

```
# 4) Spearman correlation of water level with water temperature
v = sub[["water_level_m", "water_temp_C"]].dropna()
sp = stats.spearmanr(v["water_level_m"], v["water_temp_C"])
print(f"4) Spearman(level, temperature) = {sp.statistic:.2f}, p = {sp.pvalue:.2e}, n = {len(v)}\n")
```

```
4) Spearman(level, temperature) = -0.61, p = 9.33e-09, n = 74
```

```
# 5) normality of the anomalies (Shapiro-Wilk)
sw = stats.shapiro(sub["anom"])
print(f"5) Shapiro-Wilk: W = {sw.statistic:.3f}, p = {sw.pvalue:.4f} ->",
      "normal" if sw.pvalue > 0.05 else "not normal", "\n")
```

```
5) Shapiro-Wilk: W = 0.958, p = 0.0017 -> not normal
```

```
# 6) outliers (|z| > 3)
z = (sub["anom"] - sub["anom"].mean()) / sub["anom"].std()
print("6) Number of |z|>3 outliers:", int((z.abs() > 3).sum()), "\n")
```

```
6) Number of |z|>3 outliers: 1
```

```
# 7) difference between wells (Kruskal-Wallis)
kw = stats.kruskal(*[g["water_level_m"].values for _, g in sub.groupby("well")])
print(f"7) Kruskal-Wallis across wells: H = {kw.statistic:.1f}, p = {kw.pvalue:.4f}")
```

```
7) Kruskal-Wallis across wells: H = 31.6, p = 0.0028
```